

Projeto MICROC – Etapa 1

Analizador léxico

2º semestre de 2026

Objetivo

Nesta etapa, o grupo implementará um analisador léxico que transforma o texto de um programa MICROC em uma sequência padronizada de tokens. O resultado será usado pelas etapas seguintes; portanto, nomes, números, valores e posições fazem parte de uma interface pública e não podem ser escolhidos livremente.

Resultado da etapa

arquivo .microc → texto-fonte → Lexer → sequência de Token

As regras da linguagem estão na Especificação da MICROC 2.0. Este enunciado define a interface da etapa e esclarece como essas regras devem ser observadas pelo lexer; ele não substitui a especificação da linguagem.

1 Material inicial e entrega

O starter contém:

- `Lexer.py`, com o enum, a classe imutável `Token`, a exceção `LexerError` e as assinaturas públicas do lexer;
- `runner.py`, responsável por ler um arquivo e imprimir o resultado;
- `test.microc`, um programa pequeno para experimentação; e
- `tests/test.py`, com casos públicos representativos em `pytest`;
- `README.md`, `requirements-dev.txt` e `pyproject.toml`, com as instruções e a configuração do ambiente de testes; e
- `.github/workflows/classroom.yml`, que executa os testes públicos automaticamente a cada envio ao GitHub.

O grupo deve implementar o reconhecimento em `Lexer.py`. É permitido criar módulos Python auxiliares, desde que `from Lexer import Lexer, LexerError, Token, TokenKind` continue funcionando e o `runner` fornecido não precise ser alterado. Os nomes, números e campos das classes públicas não devem ser mudados.

O grupo deve estar formado tanto no Canvas quanto no GitHub Classroom. O trabalho será entregue integralmente no repositório criado pelo GitHub Classroom; arquivos enviados pelo Canvas ou por outro canal não substituem essa entrega. Os pesos dos testes serão publicados no Canvas e no próprio Classroom.

Prazo desta etapa

A entrega do lexer encerra-se no **domingo, 6 de setembro de 2026, às 23h59**, no horário de Brasília. O commit considerado na correção deve estar no repositório correto do GitHub Classroom até esse horário.

2 Contrato dos tokens

2.1 Enumeração obrigatória

`TokenKind` possui exatamente os membros abaixo. Os intervalos foram separados por categoria para tornar diagnósticos e testes mais legíveis.

Nome	Número	Lexema ou categoria
EOF	-1	fim da entrada
IDENTIFIER	1	identificador
INT_LITERAL	2	literal inteiro decimal
STRING_LITERAL	3	literal de string
KW_INT	10	int
KW_BOOL	11	bool
KW_VOID	12	void
KW_TRUE	13	true
KW_FALSE	14	false
KW_IF	15	if
KW_ELSE	16	else
KW_WHILE	17	while

Nome	Número	Lexema ou categoria
KW_RETURN	18	return
KW_PRINT	19	print
PLUS	20	+
MINUS	21	-
STAR	22	*
SLASH	23	/
PERCENT	24	%
LESS	25	<
LESS_EQUAL	26	<=
GREATER	27	>
GREATER_EQUAL	28	>=
EQUAL_EQUAL	29	==
NOT_EQUAL	30	!=
LOGICAL_AND	31	&&
LOGICAL_OR	32	
LOGICAL_NOT	33	!
ASSIGN	34	=
LEFT_PAREN	40	(
RIGHT_PAREN	41	)
LEFT_BRACE	42	{
RIGHT_BRACE	43	}
COMMA	44	,
SEMICOLON	45	;

Não existem membros públicos para espaços, quebras de linha ou comentários. Esses elementos são consumidos e descartados antes de o próximo token ser produzido.

2.2 Estrutura de Token

A classe fornecida é uma `dataclass` imutável:

```
@dataclass(frozen=True)
class Token:
    kind: TokenKind
    lexeme: str
    value: int | str | bool | None
    line: int
    column: int
```

`lexeme` preserva exatamente a grafia do token no fonte. `value` contém sua interpretação quando ela é útil:

Categoria	Valor
IDENTIFIER	o próprio nome, como 'contador'
INT_LITERAL	o inteiro Python correspondente, preservando zeros somente no lexema
STRING_LITERAL	conteúdo decodificado, sem aspas
KW_TRUE / KW_FALSE	True / False
demais tokens	None

Uma sequência decimal maior que $2^{63} - 1$ continua sendo um `INT_LITERAL` e pode ser convertida para um inteiro Python. A validação do limite da linguagem será feita na análise semântica, não nesta etapa.

2.3 Posições

Linha e coluna começam em 1 e indicam o primeiro caractere do lexema. Cada caractere de espaço ou tabulação avança uma coluna. Uma quebra `\n` avança a linha e reinicia a coluna em 1. O runner lê arquivos em modo texto, normalizando as terminações de linha usuais.

O EOF possui lexema vazio, valor `None` e a posição imediatamente posterior ao último caractere. Assim, a entrada vazia termina em 1:1 e uma entrada que termina com uma quebra de linha termina na linha seguinte, coluna 1.

3 Regras de reconhecimento

3.1 Maior prefixo e prioridade

Em cada posição, deve ser consumido o maior prefixo que forme um token válido. Por isso, `<=`, `>=`, `==`, `!=`, `&&` e `||` têm prioridade sobre seus prefixos de um caractere.

Identificadores seguem `[A-Za-z_][A-Za-z0-9_]*`. Depois de consumir o identificador completo, o lexer consulta a tabela de palavras reservadas. Há distinção entre maiúsculas e minúsculas: `while` é reservado, mas `While` é `IDENTIFIER`.

Literais inteiros contêm somente algarismos decimais. O sinal não pertence ao literal; `-10` produz `MINUS` e `INT_LITERAL`. Zeros à esquerda não mudam a base. A forma `0b101` não é um literal especial: lexicalmente, ela produz o inteiro 0 seguido do identificador `b101`.

3.2 Espaços e comentários

Espaços, tabulações e quebras de linha separam tokens e são descartados. Comentários de linha começam com `//`; comentários de bloco começam com `/*` e terminam no primeiro `*/`. Comentários de bloco não são aninhados.

O fim do arquivo dentro de comentário de bloco é erro léxico. O conteúdo descartado ainda deve atualizar linha e coluna, e caracteres não ASCII continuam inválidos mesmo quando aparecem dentro de comentários.

3.3 Strings

O lexema de `STRING_LITERAL` inclui as aspas e as sequências de escape. O valor exclui as aspas e decodifica exatamente:

Grafia	Conteúdo
<code>\n</code>	quebra de linha
<code>\t</code>	tabulação
<code>\"</code>	aspa dupla
<code>\\</code>	barra invertida

Qualquer outro escape é erro léxico. Também é erro encontrar quebra de linha ou EOF antes da aspa final. Marcadores de comentário dentro de uma string são apenas conteúdo.

O lexer produz um token para cada literal. Em:

```

1 print (
2     "resultado "
3     "final = ",
4     x
5 );
```

há dois tokens `STRING_LITERAL`. A concatenação será implementada pelo parser em uma etapa posterior.

3.4 Caracteres inválidos

O arquivo-fonte da MICROC é ASCII. Caractere não ASCII, caractere que não inicia token e ocorrências isoladas de `&` ou `|` são erros léxicos. O lexer não deve ignorar o caractere nem continuar silenciosamente.

4 Erros

Erros devem ser representados por `LexerError`. A exceção possui os atributos públicos `message`, `line` e `column`. Sua representação textual possui a forma:

```
erro léxico em linha:coluna: descrição
```

Os testes verificam obrigatoriamente a classe e a posição do erro. A redação da descrição pode variar, exceto quando um teste de interface declarar uma saída completa. Para string não terminada por EOF e comentário de bloco não terminado, a posição informada é a do delimitador inicial. Para quebra de linha em string, a posição é a própria quebra; para escape inválido, é a barra invertida.

O runner primeiro conclui `Lexer(source).scan()`. Se houver erro, ele não imprime tokens parciais, escreve o diagnóstico em `stderr` e termina com status 1. Erros de uso ou leitura terminam com status 2.

5 Estratégias permitidas

O resultado final é fixo, mas a organização interna é escolha do grupo. São aceitas:

- **manual:** decisões e laços explícitos para cada categoria;
- **dirigida por tabela:** estados e transições de um autômato representados por estruturas de dados; e
- **mista:** autômatos para categorias regulares e rotinas manuais para comentários, strings ou outros casos convenientes.

Não é permitido usar o gerador de lexer do SLY, PLY ou ferramenta equivalente. Também não é aceita uma implementação que delegue todo o reconhecimento a uma coleção global de expressões regulares. A biblioteca padrão pode ser usada para tarefas auxiliares, mas o grupo deve implementar a escolha, o consumo e o avanço dos tokens.

O starter não fornece `peek`, `advance`, estados ou tabela de transições. O grupo pode criar os auxiliares adequados à estratégia escolhida.

6 Saída Aceita

O comando de referência é:

```
python runner.py arquivo.microc
```

Cada token ocupa uma linha:

```
<numero, NOME, repr(lexeme), repr(value), linha, coluna>
```

`repr` não é um campo armazenado. Ele é usado somente na impressão para manter aspas, barras, tabs e quebras de linha visíveis em uma única linha. Por exemplo, o programa:

```
1 int x = 42;
```

produz:

```
<10, KW_INT, 'int', None, 1, 1>
<1, IDENTIFIER, 'x', 'x', 1, 5>
<34, ASSIGN, '=', None, 1, 7>
<2, INT_LITERAL, '42', 42, 1, 9>
<45, SEMICOLON, ';', None, 1, 11>
<-1, EOF, '', None, 2, 1>
```

Para o texto de origem `"a\n"`, o lexema conserva a barra e o valor contém uma quebra real. O uso de `repr` impede que essa quebra divida a representação do token em duas linhas.

7 Testes e avaliação

Execute os testes públicos a partir do diretório do starter:

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
```

A correção utilizará esses casos e testes ocultos adicionais. Cada teste terá peso publicado e concederá todos ou nenhum de seus pontos. Casos ocultos não alteram a especificação descrita neste documento; apenas exercitam outras entradas.

Serão avaliados, entre outros:

- `enum` e estrutura imutável dos tokens;
- palavras reservadas, identificadores, inteiros e operadores;
- maior prefixo e distinção entre maiúsculas e minúsculas;
- descarte de espaços e comentários com posições corretas;
- strings, escapes, grafia original e valor decodificado;
- erros léxicos e suas coordenadas;
- token EOF, formato da saída e códigos de encerramento; e
- comportamento em arquivos vazios e casos de borda.

8 Checklist de entrega

Antes do prazo, verifique se:

1. o projeto executa com a versão de Python indicada no Classroom;
2. `TokenKind`, `Token` e o formato da saída não foram alterados;
3. o lexer recebe texto, não um caminho de arquivo;
4. espaços e comentários não aparecem no fluxo público;
5. todos os caminhos terminam em um único EOF;
6. erros não deixam tokens parciais em `stdout`;
7. os testes públicos passam; e
8. somente arquivos necessários foram enviados ao repositório.